

1 THE CORE PROBLEM TO FIX

The current output is just **colored boxes with bullet points**. That is NOT acceptable.

Every slide must look like a real medical education slide — with actual drawn diagrams, shapes, connecting arrows, and visual elements. Not placeholder text. Not a description of a diagram. An **actual rendered visual**.

■ *If any slide shows only text with no real visual diagram rendered — it is INCOMPLETE. Fix it before delivering.*

2 DIAGRAM RULES — NON-NEGOTIABLE

For every slide, you must render a real diagram using **one of these three approaches**:

Option A — Mermaid.js (flowcharts, pathways, mind maps)

```
import mermaid from 'mermaid'

// Example: Visual Pathway slide
const chart = `
graph TD
  A[Retina] --> B[Optic Nerve]
  B --> C[Optic Chiasm]
  C --> D[Optic Tract]
  D --> E[LGN]
  E --> F[Visual Cortex]
`

// Render as actual SVG inside each slide — NOT as text
```

Option B — React Flow (node-graph mind maps)

```
npm install reactflow

import ReactFlow from 'reactflow'
import 'reactflow/dist/style.css'

// Render nodes and edges as a real interactive graph
const nodes = [
  { id: '1', position: { x: 0, y: 0 }, data: { label: 'Human Eye' } },
  { id: '2', position: { x: -200, y: 100 }, data: { label: 'Anatomy' } },
  { id: '3', position: { x: 0, y: 100 }, data: { label: 'Optics' } },
  { id: '4', position: { x: 200, y: 100 }, data: { label: 'Visual Pathway' } },
]
const edges = [
  { id: 'e1-2', source: '1', target: '2' },
  { id: 'e1-3', source: '1', target: '3' },
  { id: 'e1-4', source: '1', target: '4' },
]
<ReactFlow nodes={nodes} edges={edges} fitView />
```

Option C — Hand-drawn SVG (anatomy diagrams)

```
// Example: Eye cross-section
<svg viewBox='0 0 300 200' xmlns='http://www.w3.org/2000/svg'>
  {/ * Sclera - outer white */}
  <ellipse cx='150' cy='100' rx='120' ry='90'
  fill='#fff3e0' stroke='#333' strokeWidth='2' />

  {/ * Cornea - front transparent dome */}
  <path d='M 50 100 Q 30 60 50 30'
  fill='none' stroke='#90CAF9' strokeWidth='3' />

  {/ * Iris */}
  <circle cx='150' cy='100' r='40'
  fill='#b3e5fc' stroke='#333' strokeWidth='1.5' />

  {/ * Pupil */}
  <circle cx='150' cy='100' r='20' fill='#1a237e' />

  {/ * Lens */}
  <ellipse cx='160' cy='100' rx='15' ry='30'
  fill='rgba(200,230,255,0.6)' stroke='#555' strokeWidth='1' />

  {/ * Retina - back inner lining */}
  <ellipse cx='155' cy='100' rx='100' ry='75'
  fill='none' stroke='#EF9A9A' strokeWidth='2' strokeDasharray='4' />

  {/ * Optic nerve */}
  <line x1='265' y1='100' x2='290' y2='100'
  stroke='#FFF176' strokeWidth='4' />

  {/ * Labels */}
  <text x='25' y='25' fontSize='9' fill='#333'>Cornea</text>
  <text x='125' y='55' fontSize='9' fill='#333'>Iris</text>
  <text x='155' y='75' fontSize='9' fill='#333'>Lens</text>
  <text x='60' y='170' fontSize='9' fill='#333'>Sclera</text>
  <text x='120' y='185' fontSize='9' fill='#EF9A9A'>Retina</text>
  <text x='265' y='92' fontSize='9' fill='#FFF176'>Optic N.</text>
</svg>
```

■ **KEY NOTE:** You *MUST* use one of these three approaches on *EVERY* slide. No exceptions. No placeholder boxes.

3 SLIDE LAYOUT — EXACT REACT COMPONENT

Build every slide using this exact React component structure. Copy this precisely:

```
// SlideLayout.jsx
export default function SlideLayout({ slide, n, total, conceptNodes }) {
  return (
    <div style={{
      background: 'linear-gradient(135deg, #0A0F2C 0%, #1a1f4e 100%)',
      minHeight: '100vh',
      display: 'flex',
      flexDirection: 'column',
      fontFamily: "'Inter', 'DM Sans', sans-serif",
    }}>

      {/* ■■ TOP BAR ■■ */}
      <div style={{
        background: '#00BCD4',
        padding: '8px 24px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
      }}>
        <span style={{ color: 'white', fontWeight: 'bold', fontSize: '12px' }}>
          MEDICAL EDUCATION | SYNAPSE
        </span>
        <span style={{ color: 'white', fontSize: '12px' }}>
          Slide {n} / {total}
        </span>
      </div>

      {/* ■■ MAIN CONTENT: LEFT = text, RIGHT = diagram ■■ */}
      <div style={{
        display: 'grid',
        gridTemplateColumns: '1fr 1fr',
        flex: 1,
        gap: '24px',
        padding: '32px',
      }}>

        {/* LEFT PANEL */}
        <div>
```

```
<h2 style={{
  color: '#00BCD4',
  fontSize: '22px',
  fontWeight: '800',
  marginBottom: '16px',
  fontFamily: "'Space Grotesk', sans-serif",
}}>
{slide.title}
</h2>

{slide.bullets.map((b, i) => (
  <div key={i} style={{ display: 'flex', gap: '10px', marginBottom: '12px' }}>
    <span style={{ color: '#00BCD4', fontWeight: 'bold', fontSize: '16px' }}>
      &#9658;
    </span>
    <p style={{ color: 'white', fontSize: '14px', lineHeight: '1.6', margin: 0 }}>
      {b}
    </p>
  </div>
))}

{/* KEY INSIGHT BOX */}
<div style={{
  borderLeft: '4px solid #00BCD4',
  background: 'rgba(0,188,212,0.1)',
  padding: '12px 16px',
  marginTop: '20px',
  borderRadius: '0 8px 8px 0',
}}>
  <span style={{
    color: '#00BCD4',
    fontWeight: 'bold',
    fontSize: '12px',
    letterSpacing: '0.05em',
  }}>
    &#9889; KEY INSIGHT
  </span>
  <p style={{
```

```

color: '#e0f7fa',
fontStyle: 'italic',
fontSize: '13px',
margin: '4px 0 0',
}}>
{slide.keyInsight}
</p>
</div>
</div>

{/* RIGHT PANEL — DIAGRAM */}
<div style={{
background: 'rgba(255,255,255,0.05)',
borderRadius: '16px',
border: '1px solid rgba(0,188,212,0.3)',
padding: '20px',
display: 'flex',
flexDirection: 'column',
alignItems: 'center',
justifyContent: 'center',
}}>
<p style={{ color: '#00BCD4', fontSize: '11px', fontWeight: 'bold',
textTransform: 'uppercase', marginBottom: '12px' }}>
{slide.diagramType === 'flowchart' ? 'Mechanism Flowchart'
: slide.diagramType === 'anatomy' ? 'Anatomy Diagram'
: slide.diagramType === 'mindmap' ? 'Mind Map'
: 'Clinical Pathway'}
</p>
{/* RENDER DIAGRAM HERE */}
<DiagramComponent
type={slide.diagramType}
data={slide.diagramData}
/>
</div>
</div>

{/* ■■ BOTTOM: CONCEPT MAP STRIP ■■ */}
<div style={{

```

```
background: 'rgba(0,0,0,0.35)',
padding: '10px 32px',
display: 'flex',
gap: '6px',
alignItems: 'center',
flexWrap: 'wrap',
}}>
{conceptNodes.map((node, i) => (
<React.Fragment key={i}>
<div style={{
background: 'rgba(0,188,212,0.2)',
border: '1px solid #00BCD4',
borderRadius: '20px',
padding: '4px 12px',
color: 'white',
fontSize: '11px',
}}>
{node}
</div>
{i < conceptNodes.length - 1 && (
<span style={{ color: '#00BCD4', fontSize: '14px' }}>&#8594;</span>
)}
</React.Fragment>
)}}
</div>
</div>
)
}
```

4 DiagramComponent — Router + Mermaid Renderer

```
// DiagramComponent.jsx
import MermaidDiagram from './MermaidDiagram'
import AnatomySVG from './AnatomySVG'
import MindMapFlow from './MindMapFlow'
import PathwaySVG from './PathwaySVG'
import DiagramTable from './DiagramTable'

export default function DiagramComponent({ type, data }) {
  switch (type) {
    case 'flowchart': return <MermaidDiagram chart={data.mermaid} />
    case 'anatomy': return <AnatomySVG topic={data.topic} />
    case 'mindmap': return <MindMapFlow nodes={data.nodes} edges={data.edges} />
    case 'pathway': return <PathwaySVG steps={data.steps} />
    case 'table': return <DiagramTable rows={data.rows} />
    default: return null
  }
}
```

MermaidDiagram.jsx — Dark-themed Mermaid renderer

```
// MermaidDiagram.jsx
import mermaid from 'mermaid'
import { useEffect, useRef } from 'react'

mermaid.initialize({
  startOnLoad: false,
  theme: 'dark',
  themeVariables: {
    primaryColor: '#00BCD4',
    primaryTextColor: 'ffffff',
    primaryBorderColor: '#007C91',
    lineColor: '#00BCD4',
    secondaryColor: '#1a1f4e',
    tertiaryColor: '#0A0F2C',
    background: '#0A0F2C',
    nodeBorder: '#00BCD4',
    clusterBkg: '#1a1f4e',
    titleColor: '#00E5FF',
    edgeLabelBackground: '#0A0F2C',
  },
})

export default function MermaidDiagram({ chart }) {
  const ref = useRef(null)
  const id = useRef(`mermaid-${Math.random().toString(36).slice(2)}`)

  useEffect(() => {
    if (!ref.current || !chart) return
    mermaid
      .render(id.current, chart)
      .then(({ svg }) => { ref.current.innerHTML = svg })
      .catch(console.error)
  }, [chart])

  return (
    <div
      ref={ref}
      style={{ width: '100%', minHeight: '200px' }}
    />
  )
}
```

```
/>  
)  
}
```


5 Human Eye — Pre-built SVGs & Mermaid Charts

Since the uploaded topic is **Human Eye**, pre-build these specific diagrams for each slide:

Slide 2 — Eye Cross-Section Anatomy SVG

```
// AnatomySVG.jsx — eye cross-section
export function EyeAnatomySVG() {
  return (
    <svg viewBox='0 0 320 220' width='100%' xmlns='http://www.w3.org/2000/svg'>
      { /* Sclera */ }
      <ellipse cx='155' cy='110' rx='130' ry='95' fill='#fff8f0' stroke='#555' strokeWidth='2' />
      { /* Choroid */ }
      <ellipse cx='155' cy='110' rx='122' ry='87' fill='none' stroke='#c62828' strokeWidth='4'
        strokeDasharray='6 3' />
      { /* Retina */ }
      <ellipse cx='155' cy='110' rx='113' ry='79' fill='none' stroke='#EF9A9A' strokeWidth='2' />
      { /* Vitreous */ }
      <ellipse cx='168' cy='110' rx='90' ry='72' fill='rgba(200,230,255,0.15)' stroke='none' />
      { /* Lens */ }
      <ellipse cx='118' cy='110' rx='14' ry='28' fill='rgba(176,224,255,0.7)' stroke='#4FC3F7'
        strokeWidth='1.5' />
      { /* Iris */ }
      <circle cx='95' cy='110' r='38' fill='#5C6BC0' stroke='#1a237e' strokeWidth='2' />
      { /* Pupil */ }
      <circle cx='95' cy='110' r='18' fill='#0D0D0D' />
      { /* Cornea */ }
      <path d='M 35 68 Q 10 110 35 152' fill='none' stroke='#90CAF9' strokeWidth='4' />
      { /* Optic disc */ }
      <circle cx='275' cy='110' r='10' fill='#FFF176' stroke='#F9A825' strokeWidth='1.5' />
      { /* Optic nerve */ }
      <rect x='283' y='104' width='28' height='12' fill='#FFF176' rx='4' />
      { /* Macula */ }
      <circle cx='235' cy='110' r='14' fill='none' stroke='#FF8A65' strokeWidth='2' />
      { /* Fovea */ }
      <circle cx='235' cy='110' r='5' fill='#FF7043' />

      { /* Labels */ }
      <text x='5' y='30' fontSize='9' fill='#90CAF9' fontWeight='bold'>Cornea</text>
      <text x='60' y='58' fontSize='9' fill='#7986CB'>Iris</text>
      <text x='108' y='78' fontSize='9' fill='#4FC3F7'>Lens</text>
      <text x='155' y='52' fontSize='9' fill='#EF9A9A'>Retina</text>
      <text x='60' y='200' fontSize='9' fill='#BCAAA4'>Sclera</text>
      <text x='215' y='95' fontSize='9' fill='#FF8A65'>Macula</text>

      <text x='253' y='95' fontSize='9' fill='#FFF176'>Optic Disc</text>
      <text x='285' y='95' fontSize='9' fill='#FFF176'>Optic N.</text>
    </svg>
  )
}
```

Slide 3 — Aqueous Humor Flow (Mermaid)

```

const aqueous_chart = `
flowchart TD
A([Ciliary Body\nProduction]) -->|2-3 uL/min| B[Posterior Chamber]
B -->|Through Pupil| C[Anterior Chamber]
C -->|80-90%| D[Trabecular Meshwork]
C -->|10-20%| E[Uveoscleral Pathway]
D --> F([Episcleral Veins])
E --> F
style A fill:#00BCD4,color:#fff
style F fill:#007C91,color:#fff
style D fill:#1a1f4e,color:#fff
`

```

Slide 5 — Phototransduction Cascade (Mermaid)

```

const photo_chart = `
flowchart TD
A([Photon Absorbed]) --> B[Opsin Activated]
B --> C[Transducin G-protein]
C --> D[PDE Activated]
D --> E[cGMP Decreased]
E --> F[Cation Channels Close]
F --> G[Photoreceptor Hyperpolarizes]
G --> H([Glutamate Release Reduced])
H --> I[Signal to Bipolar Cell]
style A fill:#00BCD4,color:#fff
style H fill:#004D61,color:#fff
style I fill:#007C91,color:#fff
`

```

Slide 6 — Visual Pathway SVG

```

export function VisualPathwaySVG() {
  const box = (x,y,w,h,label,color='#1a1f4e') => (
    <g key={label}>
      <rect x={x} y={y} width={w} height={h} rx='6'
        fill={color} stroke='#00BCD4' strokeWidth='1.5' />
      <text x={x+w/2} y={y+h/2+4} textAnchor='middle'
        fontSize='9' fill='white' fontWeight='bold'>{label}</text>
    </g>
  )
  const arr = (x1,y1,x2,y2) => (
    <line x1={x1} y1={y1} x2={x2} y2={y2}
      stroke='#00BCD4' strokeWidth='1.5'
      markerEnd='url(#arrow)' />
  )
  return (
    <svg viewBox='0 0 320 260' width='100%'>
      <defs>
        <marker id='arrow' markerWidth='8' markerHeight='8'
          refX='6' refY='3' orient='auto'>
          <path d='M0,0 L0,6 L8,3 z' fill='#00BCD4' />
        </marker>
      </defs>
      /* Left Eye */
      {box(10, 10, 60, 24, 'L Eye', '#00BCD4')}
      /* Right Eye */
      {box(250, 10, 60, 24, 'R Eye', '#00BCD4')}
      /* Optic nerves */
      {arr(40, 34, 120, 90)}
      {arr(280, 34, 200, 90)}
      /* Chiasm */
      {box(110, 90, 100, 28, 'Optic Chiasm')}
      /* Crossed nasal fibers */
      <path d='M 120,118 Q 160,145 200,118'
        fill='none' stroke='#FF7072' strokeWidth='1.5' strokeDasharray='4' />
      /* L Optic tract */
      {arr(120, 118, 70, 158)}
      /* R Optic tract */
      {arr(200, 118, 250, 158)}

      /* LGN */
      {box(30, 158, 80, 24, 'L LGN')}
      {box(210, 158, 80, 24, 'R LGN')}
      /* Radiations */
      {arr(70, 182, 70, 218)}
      {arr(250, 182, 250, 218)}
      /* Visual Cortex */
      {box(30, 218, 80, 28, 'L V1', '#004D61')}
      {box(210, 218, 80, 28, 'R V1', '#004D61')}
      /* Chiasm label */
      <text x='148' y='148' textAnchor='middle' fontSize='8'
        fill='#FF7072'>Nasal fibers cross</text>
    </svg>
  )
}

```

Slide 9 — Red Eye Triage Decision Flowchart (Mermaid)

```

const redEye_chart = `
flowchart TD
A([Red Eye Presentation]) --> B{Pain +\nPhotophobia?}
B -->|Yes| C[Fluorescein Staining]
B -->|No| D{Vision\nLoss?}
C --> E{Ulcer\nPresent?}
E -->|Yes| F[Bacterial Keratitis\nUrgent Abx]
E -->|No| G[Anterior Uveitis\nSlit Lamp]
D -->|Yes| H{IOP > 40?}
D -->|No| I[Conjunctivitis\nObserve]
H -->|Yes| J([ACUTE ANGLE CLOSURE\nEmergency])
H -->|No| K[RAPD Check]
K --> L[Optic Nerve\nAssessment]
style A fill:#00BCD4,color:#fff
style J fill:#c62828,color:#fff,font-weight:bold
style F fill:#e65100,color:#fff
`

```

Slide 10 — Full Mind Map (ReactFlow)

```

// MindMapFlow.jsx
import ReactFlow, { Background, Controls } from 'reactflow'
import 'reactflow/dist/style.css'

const nodeStyle = {
  background: '#1a1f4e',
  color: 'white',
  border: '2px solid #00BCD4',
  borderRadius: '12px',
  padding: '8px 14px',
  fontSize: '12px',
  fontWeight: 'bold',
}

const centerStyle = {
  ...nodeStyle,
  background: '#00BCD4',
  fontSize: '14px',
  padding: '12px 20px',
}

const nodes = [
  { id: 'c', position: {x:300,y:200}, data:{label:'Human Eye'}, style:centerStyle },
  { id: 'a1', position: {x:80, y:40}, data:{label:'Anatomy'}, style:nodeStyle },
  { id: 'a2', position: {x:520,y:40}, data:{label:'Optics'}, style:nodeStyle },
  { id: 'a3', position: {x:600,y:200}, data:{label:'Visual Pathway'}, style:nodeStyle },
  { id: 'a4', position: {x:520,y:360}, data:{label:'Emergencies'}, style:nodeStyle },
  { id: 'a5', position: {x:80, y:360}, data:{label:'Clinical Exam'}, style:nodeStyle },
  { id: 'a6', position: {x:0, y:200}, data:{label:'Disorders'}, style:nodeStyle },
  // Sub-nodes for Anatomy
  { id: 'b1', position: {x:0, y:-40}, data:{label:'Cornea / Sclera'},
    style:{...nodeStyle,fontSize:'10px'} },
  { id: 'b2', position: {x:80, y:-60}, data:{label:'Uvea / Retina'},
    style:{...nodeStyle,fontSize:'10px'} },
  { id: 'b3', position: {x:160,y:-40}, data:{label:'Lens / Vitreous'},
    style:{...nodeStyle,fontSize:'10px'} },
]

const edges = [
  { id: 'e-c-a1', source: 'c', target: 'a1', style:{stroke:'#00BCD4'} },
  { id: 'e-c-a2', source: 'c', target: 'a2', style:{stroke:'#00BCD4'} },

```

```
{ id:'e-c-a3', source:'c', target:'a3', style:{stroke:'#00BCD4'} },
{ id:'e-c-a4', source:'c', target:'a4', style:{stroke:'#00BCD4'} },
{ id:'e-c-a5', source:'c', target:'a5', style:{stroke:'#00BCD4'} },
{ id:'e-c-a6', source:'c', target:'a6', style:{stroke:'#00BCD4'} },
{ id:'e-a1-b1', source:'a1', target:'b1', style:{stroke:'#4FC3F7',strokeDasharray:'4'} },
{ id:'e-a1-b2', source:'a1', target:'b2', style:{stroke:'#4FC3F7',strokeDasharray:'4'} },
{ id:'e-a1-b3', source:'a1', target:'b3', style:{stroke:'#4FC3F7',strokeDasharray:'4'} },
]

export default function MindMapFlow() {
  return (
    <div style={{ width:'100%', height:'350px', background:'#0A0F2C', borderRadius:'12px' }}>
      <ReactFlow nodes={nodes} edges={edges} fitView>
        <Background color='#1a1f4e' gap={20} />
        <Controls />
      </ReactFlow>
    </div>
  )
}
```

API call — generatePresentation.js

```
// generatePresentation.js (Next.js API route / server action)
export async function generatePresentation(topic) {
  const systemPrompt = `
  You are SYNAPSE, a medical education AI.
  Generate a complete, exam-ready presentation on: ${topic}

  Return ONLY a valid JSON object – no markdown, no preamble.
  Schema:
  {
  slides: [
  {
  slideNumber: number,
  title: string,
  keyInsight: string,
  bullets: string[], // 5-6 items
  conceptMapNodes: string[], // 4-6 short labels
  diagramType: 'flowchart' | 'anatomy' | 'mindmap' | 'pathway' | 'table' | 'none',
  diagramData: {
  mermaid?: string, // Mermaid chart string (for flowchart/pathway)
  topic?: string, // anatomy topic key
  nodes?: object[], // ReactFlow nodes (for mindmap)
  edges?: object[], // ReactFlow edges (for mindmap)
  steps?: string[], // pathway steps
  rows?: string[][], // table rows
  },
  faq?: { question: string, answer: string }[],
  glossary?: { term: string, definition: string }[],
  conditions?: { name: string, features: string, clinicalClue: string }[],
  }
  ]
  }
  Rules:
  - EVERY slide with mechanism/anatomy/pathway content MUST have a real mermaid string or SVG key.
  - Slide 9 is always a full mind map (diagramType: mindmap).
  - Slide 10 is always FAQ (8-10 high-yield exam questions).
  - Slide 11 is always glossary (8-10 terms).
  - Content must be high-yield and exam-focused.
  `
}
```

```

const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 4096,
    system: systemPrompt,
    messages: [{ role: 'user', content: `Generate presentation for: ${topic}` }],
  }),
})

const data = await response.json()
const raw = data.content[0].text

try {
  return JSON.parse(raw)
} catch {
  // Strip accidental markdown fences
  const clean = raw.replace(/```json|```/g, '').trim()
  return JSON.parse(clean)
}
}

```

7 Definition of Done — Success Checklist

#	Area	Requirement
1	Slide Layout	Every slide has top bar (SYNAPSE branding + slide counter)
2	Left Panel	Title + 5-6 bullet points + teal Key Insight box on every slide
3	Right Panel	Real rendered diagram — Mermaid SVG, ReactFlow, or hand-coded SVG
4	Bottom Strip	Concept map nodes connected by teal arrows
5	Eye Slide 2	Eye cross-section SVG with all 8 labeled anatomical parts
6	Eye Slide 3	Aqueous humor Mermaid flowchart rendered as actual SVG
7	Eye Slide 5	Phototransduction cascade Mermaid chart rendered
8	Eye Slide 6	Visual pathway SVG with chiasm crossing shown
9	Eye Slide 9	Red eye triage Mermaid decision tree rendered
10	Eye Slide 10	ReactFlow mind map with Human Eye center + 6 branches
11	FAQ Slide	8-10 collapsible exam questions with answers
12	Glossary Slide	8-10 terms in two-column grid with numbered badges
13	Navigation	Left/right arrows + dot indicator working
14	Responsive	Works on mobile and desktop
15	PDF Export	html2canvas + jsPDF export button functional (bonus)

■ If any slide shows *ONLY* text with no rendered diagram — the feature is incomplete. Do NOT deliver until every slide has a real visual.